

Contrôle intermédiaire du 06/11/2025 – éléments de corrigé

Exercice 1 : Palindrome (6 points)

a) Écrire une fonction **EstUnPalindrome()** qui reçoit en paramètre un tableau de caractères et qui détermine si le mot contenu dans ce tableau est un palindrome (Vrai ou Faux).

Fonction : **EstUnPalindrome**

Entrée : - **tab** : tableau 1D de caractères

Variables : - **d, f** : entiers
- **palindrome** : booléen

Sortie : booléen

Début

```
d = 0
f = Taille(tab) - 1
palindrome = VRAI

Tant Que palindrome ET d <= f Alors
    Si tab[d] ≠ tab[f] Alors
        palindrome = FAUX
    Sinon
        d = d + 1
        f = f - 1
    Fin Si
Fin Tant Que
```

Fin

b) Justifier l'utilisation de la structure itérative (boucle) choisie.

=> La structure la plus adaptée dans ce cas est le Tant Que car dès que l'on sait que le mot n'est plus un palindrome on peut sortir de la boucle. Le Pour aurait l'inconvénient de visiter toutes les cases même si l'on sait que ce n'est pas un palindrome ou alors, pire, de provoquer une sortie prématurée avec l'horrible break.

Exercice 2 : The cameraman (14 points)

a) Écrire la procédure **Binariser()** qui binarise le tableau reçu en paramètre en fonction du seuil également reçu en paramètre (4 points).

Procédure : **Binariser**

Entrée : - **tab** : tableau 2D d'entiers
- **seuil** : entier

Variables : - **i, j** : entiers

Sortie : /

Début

```
Pour i de 0 à TailleLigne(tab) - 1 Faire
    Pour j de 0 à TailleColonne(tab) - 1 Faire
        Si tab[i, j] < seuil Alors
            tab[i, j] = 0
        Sinon
            tab[i, j] = 255
        Fin Si
    Fin Pour
Fin Pour
```

Fin

b) Écrire la fonction **FiltrerSobel()** qui crée et retourne un nouveau tableau à partir du tableau reçu en paramètre en appliquant la méthode du filtre Sobel (9 points).

Fonction : **FiltrerSobel**

Entrée : - tab : tableau 2D d'entiers

Variables : - i, j : entiers
- newTab : tableau 2D d'entiers

Sortie : tableau 2D d'entiers

Début

```
// mettre des 0 dans les parties grisées de newTab
Pour i de 0 à TailleLigne(newTab) - 1 Faire
|   newTab[i, 0] = 0
|   newTab[i, TailleColonne(newTab) - 1] = 0
Fin Pour

// Remplissage du reste du tableau en appliquant le filtre
Pour i de 0 à TailleLigne(newTab) - 1 Faire
|   Pour j de 1 à TailleColonne(newTab) - 2 Faire
|   |   newTab[i, j] = tab[i, j + 1] - tab[i, j - 1]
|   Fin Pour
Fin Pour

Retourner newTab
```

Fin

On souhaite maintenant créer l'histogramme de l'image originale.

c) Déclarer le tableau nécessaire au stockage de cet histogramme (nom, type et capacité(s)).

=> histo : tableau 1D de 256 entiers

BONUS : Écrire la fonction **Histogramme()** qui renvoie le tableau contenant l'histogramme du tableau image passé en paramètre (+2 points).

Fonction : **Histogramme**

Entrée : - tab : tableau 2D d'entiers

Variables : - i, j : entiers
- histo : tableau 1D de 256 entiers

Sortie : tableau 1D de 256 entiers

Début

```
// initialiser le tableau histo
Pour i de 0 à 255 Faire
|   histo[i] = 0
Fin Pour

// remplir l'histogramme
Pour i de 0 à TailleLigne(tab) - 1 Faire
|   Pour j de 0 à TailleColonne(tab) - 1 Faire
|   |   histo[tab[i, j]] = histo[tab[i, j]] + 1
|   Fin Pour
Fin Pour
```

Fin